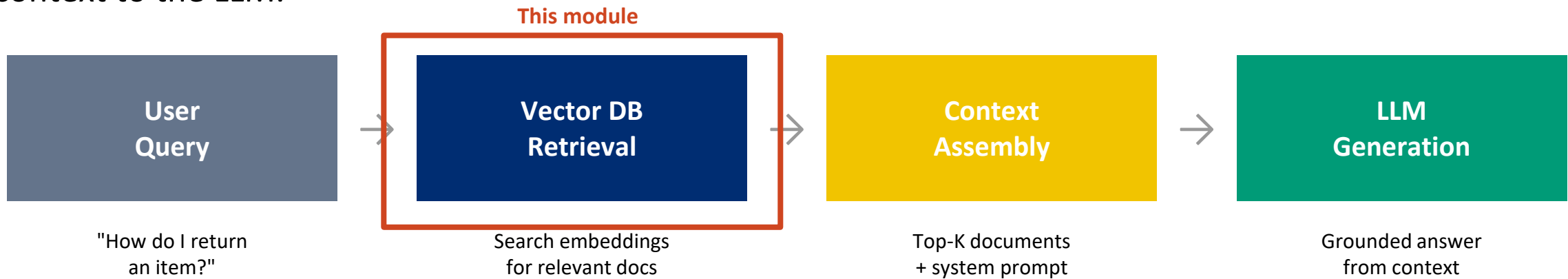


Introduction to Agentic AI

Vector Databases for AI Systems

Quick Refresher: What Is RAG?

Retrieval-Augmented Generation (RAG) grounds LLM responses in your data. Instead of relying on what the model memorized during training, RAG retrieves relevant documents at query time and feeds them as context to the LLM.



Why the Vector Database Choice Matters for RAG

The vector database determines what context the LLM sees.
Better retrieval → better answers. Bad retrieval → hallucination, regardless of how good the LLM is.

Retrieval quality depends on: chunking strategy, embedding model, index algorithm, and search method

- The database choice affects: scale ceiling, query latency, operational complexity, and cost
- You can't fix bad retrieval with a better LLM—the database is the quality bottleneck

Learning Objectives



Product Decision-Making

Evaluate six tools—Pinecone, Weaviate, ChromaDB, pgvector, Qdrant, FAISS—across scale, latency, features, ops complexity, and cost.



Product Fluency

Understand each product's architecture, including the library vs. database distinction. See working code for all six.



Pipeline Craft

Apply chunking, embedding, and CRUD patterns that directly impact retrieval quality across any product choice.



Retrieval Beyond Vectors

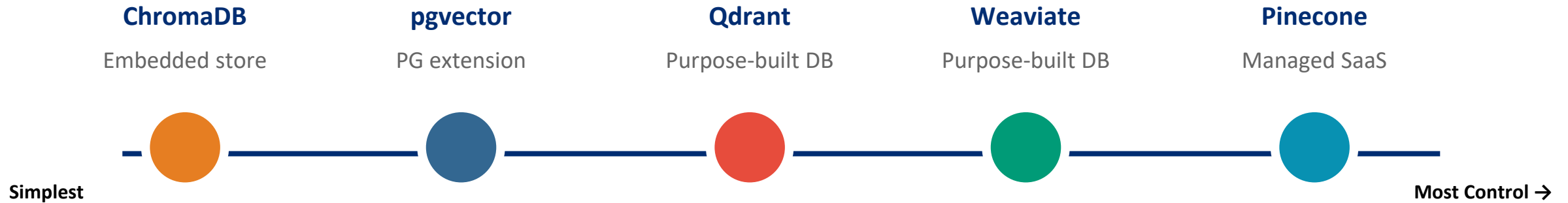
Understand hybrid search, metadata filtering, and reranking—and which products support which strategies.

01: The Decision Landscape

Why Your Choice of Vector Search Matters

Six Tools, One Spectrum

- From Embedded Library to Managed Service



FAISS—A similarity search library, not a database

From Meta AI. Computes nearest neighbors on vectors you manage yourself.

No storage, no API, no CRUD, no metadata—you build all of that.

Maximum performance + GPU acceleration. Use for billions of vectors with a strong infrastructure team.

Five Axes That Determine Your Choice



Scale

<1M: any tool works → 10–100M: Pinecone, Qdrant, Weaviate → 1B+: FAISS with IVF-PQ + GPU



Latency

Sub-5ms: FAISS in-memory → Sub-10ms: Qdrant, Pinecone → Sub-50ms: all products with tuning



Features

Pure vector → metadata filtering → hybrid search → built-in vectorization → multi-modal



Operations

Embedded (Chroma) → PG extension → Managed cloud (Pinecone) → Self-hosted → Library (FAISS)



Cost

Free (ChromaDB, pgvector, FAISS) → Usage-based SaaS (Pinecone) → Self-hosted compute costs

02: Six Products, Six Design Philosophies

Pinecone—Managed, Zero-Ops, Purpose-Built

What It Is

- Fully managed vector database.
- No infrastructure, no clusters, no tuning.
- Serverless scales automatically.
- Proprietary—no self-hosting.

Sweet Spot

- Zero operational burden
- Production RAG, fast time-to-market
- Sparse+dense hybrid search
- Same product prototype → production

Watch Out

- Vendor lock-in: proprietary API
- Cost at scale: per-read/write-unit can reach thousands/mo at 50M+
- Limited configurability

```
from pinecone import Pinecone

pc = Pinecone(api_key="...")
index = pc.Index("products")

index.upsert(vectors=[{
    "id": "doc-42-chunk-0",
    "values": embedding,
    "metadata": {"source": "manual", "category": "install"}
}])

results = index.query(
    vector=query_embedding, top_k=10,
    filter={"category": {"$eq": "install"}}
)
```

Weaviate—Hybrid Search, Modules, Multi-Modal

What It Is

- Open-source vector DB with modules.
- Native BM25 + vector hybrid search.
- Multi-modal (text + image).
- Self-hosted or Weaviate Cloud.

Sweet Spot

- Hybrid search as first-class feature
- Multi-modal (text + image)
- Built-in vectorization via modules
- Open-source + managed cloud

Watch Out

- Resource-heavy self-hosting
- Module config complexity
- Schema rigidity: class-based
- Schema changes painful at scale

```
import weaviate
from weaviate.classes.config import Configure

client = weaviate.connect_to_local()
collection = client.collections.create(
    name="Document",
    vectorizer_config=Configure.Vectorizer.text2vec_openai(),
)
collection.data.insert({"text": "Install guide...", "category": "setup"})

results = collection.query.hybrid(
    query="how to install", limit=10,
    alpha=0.5 # 0=keyword, 1=vector
)
```

ChromaDB—Local-First, Embedded, Developer-Friendly

What It Is

- Open-source embedding database.
- In-process or lightweight client-server.
- pip install and go.
- Stores original text alongside vectors.

Sweet Spot

- Prototyping in 30 seconds
- Small datasets (<1M vectors)
- Education and tutorials
- Stores raw docs with embeddings

Watch Out

- No horizontal scaling
- Not production-hardened at scale
- Migration cliff to production

```
import chromadb

client = chromadb.Client() # or PersistentClient("./data")
collection = client.get_or_create_collection("documents")

collection.upsert(
    ids=["doc-42-chunk-0"],
    embeddings=[embedding],
    documents=["The installation process requires..."],
    metadatas=[{"source": "manual", "chunk_index": 0}]
)

results = collection.query(
    query_embeddings=[query_embedding], n_results=5
)
```

pgvector—Vector Search Where Your Data Already Lives

What It Is

- PostgreSQL extension for vectors.
- Vector columns, indexes, distance operators.
- Vectors join with relational data, same transactions, same backups.

Sweet Spot

- Teams already on PostgreSQL
- Vectors JOIN with relational data
- One database to manage
- Up to ~50M vectors single node
- All Postgres tooling applies

Watch Out

- Single-node ceiling
- Can't match dedicated engines at extreme scale
- HNSW builds slow on large tables

```
-- Enable extension
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE documents ADD COLUMN embedding vector(1536);

-- Create HNSW index
CREATE INDEX ON documents
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 200);

-- Vector search + relational filter in one query
SELECT id, title, 1 - (embedding <=> $1) AS similarity
FROM documents
WHERE category = 'technical'
ORDER BY embedding <=> $1 LIMIT 10;
```

Qdrant—Rust Performance, Filtering, Production-Grade

What It Is

- Open-source vector engine in Rust.
- Horizontal scaling, on-disk storage,
- most expressive metadata filtering.
- Docker/K8s or Qdrant Cloud.

Sweet Spot

- Most expressive filtering
- High-performance P99 latency
- Multi-tenant RAG isolation
- Sparse+dense hybrid search
- On-disk for large datasets

Watch Out

- Smaller ecosystem
- Distributed setup needs K8s
- No built-in vectorization (separation of concerns by design)

```
from qdrant_client import QdrantClient, models

client = QdrantClient(url="http://localhost:6333")

client.upsert(collection_name="documents", points=[
    models.PointStruct(
        id=42, vector=embedding, # int or UUID
        payload={"source": "manual", "category": "install",
            "access_level": 2, "tags": ["setup"]}
    )
])

results = client.query_points(
    collection_name="documents", query=query_embedding,
    query_filter=models.Filter(must=[
        models.FieldCondition(key="access_level",
            range=models.Range(lte=3)),
    ]), limit=10)
```

FAISS—Raw Performance Library for Massive Scale

What It Is

- Open-source library from Meta AI.
- NOT a database. Nearest neighbors—fast.
- No storage, API, CRUD, metadata.
- You wrap it. Native CUDA GPU.

Sweet Spot

- Billions of vectors with IVF-PQ
- Maximum control over indexes
- GPU-accelerated (10–100×)
- Widest algorithm selection
- Teams with strong infra engineering

Watch Out

- Not a database: no persistence, metadata, CRUD, or API
- Build everything yourself
- Python/C++ only — no REST API
- Overkill if <50M vectors

```
import faiss, numpy as np

dimension = 1536
quantizer = faiss.IndexFlatL2(dimension)
index = faiss.IndexIVFPQ(
    quantizer, dimension,
    nlist=4096, m=64, nbits=8
)

training_data = np.random.rand(100_000, dimension).astype('float32')
index.train(training_data)
index.add(vectors) # just vectors — no metadata

index.nprobe = 64 # recall/speed tradeoff
distances, ids = index.search(query.reshape(1,-1), k=10)

gpu_index = faiss.index_cpu_to_all_gpus(index)
```

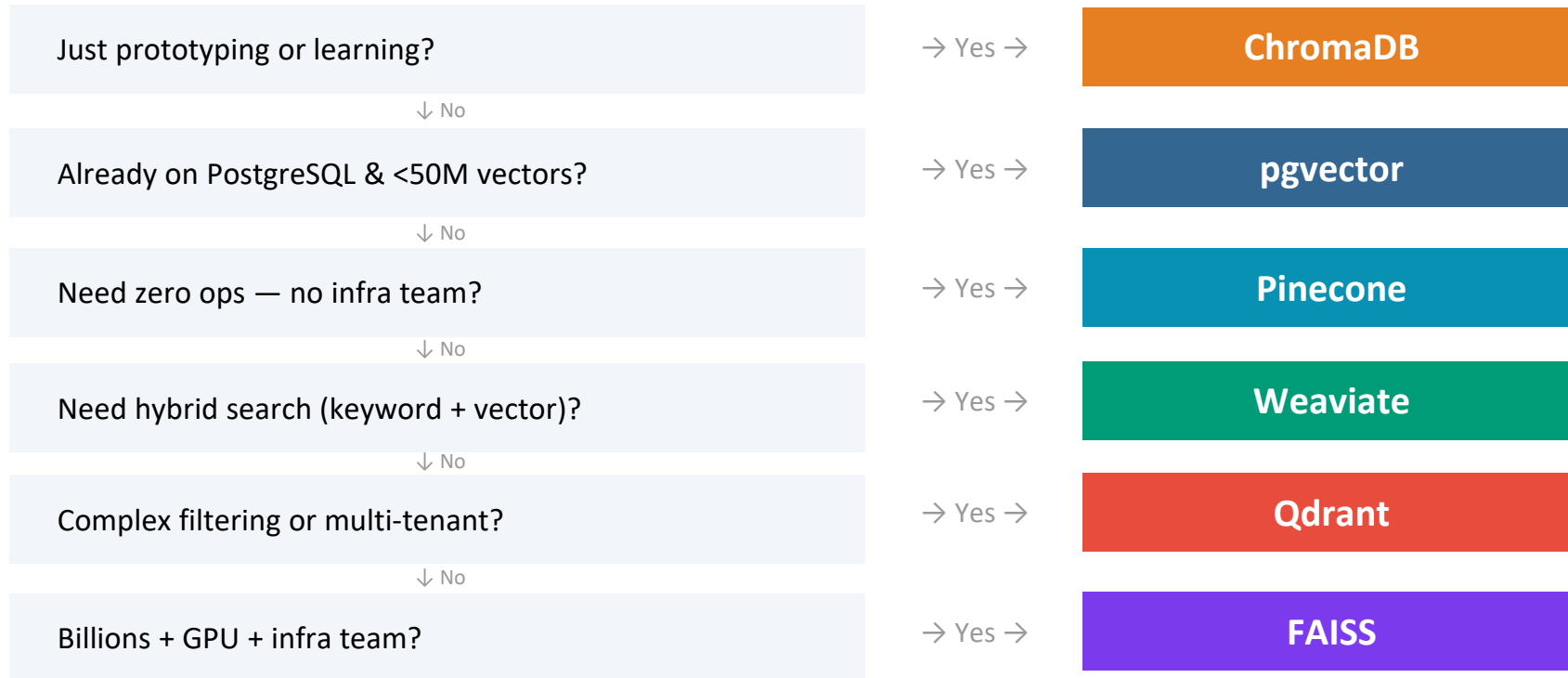
03: Head-to-Head: Choosing the Right Tool

What Goes In Matters More Than Which Database

Head-to-Head Across Key Dimensions

	Pinecone	Weaviate	ChromaDB	pgvector	Qdrant	FAISS
Type	Managed DB	Open-source DB	Embedded store	PG extension	Open-source DB	Search library
Best for	Zero-ops production	Hybrid + multi-modal	Prototyping	Teams on Postgres	Filtering + perf	Billion-scale GPU
Scale ceiling	Billions	~1B	~1M	~50M	Billions	Billions
Hybrid search	Sparse + dense	BM25 + vector	No	Via full-text	Sparse + dense	No
Filtering	Good	Good	Basic	Full SQL power	Best (most expressive)	None (build it)
Self-host	No	Yes	Yes	Yes (it's Postgres)	Yes	Yes (it's a library)
GPU support	No	No	No	No	No	Native CUDA
Key risk	Lock-in, cost	Resource-heavy	No scaling	Single-node limit	Smaller ecosystem	Build everything

The Decision Tree



04: Head-to-Head: Embeddings and Chunking

Your Database Can Only Search What Your Embeddings Capture

If you need...	Use...	Price/1M tokens
Simplest integration	OpenAI text-embedding-3-small	\$0.02
Highest retrieval quality	Voyage AI voyage-3-large	\$0.18
Best multilingual	Cohere embed-v4 or BGE-M3 (open-source)	Varies/Free
Self-hosted/data privacy	BGE-M3 on your own GPU	Free
No separate embed pipeline	Weaviate vectorizer modules	Via Weaviate

- Switching costs are high—changing models means full re-embed and re-index of everything.
- Dimensionality drives storage cost—3072-dim = 3× storage of 1024-dim at scale.
- Domain beats benchmarks—always test with your specific corpus before committing.

Chunking: Where Retrieval Quality Is Won or Lost

- *Same document, chunked differently → larger quality delta than switching databases.*

Use Case	Strategy	Size	Overlap
General RAG chatbot	Recursive (paragraph-aware)	512 tokens	50 tokens
FAQ/Q&A retrieval	Fixed-size	256 tokens	25 tokens
Legal/long-context docs	Hierarchical (section-aware)	512–1024 tokens	100–200 tokens
Code retrieval	Function/class boundaries	Natural	Minimal

Fixed-Size + Overlap

Split every N tokens. Simple and universal. May break mid-sentence—overlap mitigates.

Recursive/Semantic

Split on headings → paragraphs → sentences. Variable length. Respects content boundaries.

Document-Aware

Parse structure first, chunk within sections. Attach parent section title as metadata.

Per-Function (Code)

Use natural code boundaries. Never split mid-function. Language-aware parsers help.

05: CRUD and Metadata

The Operational Layer Most Tutorials Skip

The Update Problem and Metadata Schema

- When a source document changes, the chunk count changes.
- You can't just overwrite—you need to:
 1. Find all vectors by source_doc_id
 2. Delete old vectors
 3. Re-chunk the new document
 4. Re-embed all new chunks
 5. Insert replacement vectors

source_doc_id → bulk delete when documents change

chunk_index / total_chunks → gap detection, reassembly

tenant_id → multi-tenant data isolation (Security module)

embedding_model → know which vectors to re-embed on model upgrade

Good Metadata Schema

```
metadata = {  
    "source_doc_id": "contract-2024-0892",  
    "chunk_index": 3,  
    "total_chunks": 12,  
    "created_at": "2026-05-06",  
    "category": "legal",  
    "tenant_id": "acme-corp",  
    "embedding_model": "text-embedding-3-small",  
}
```

Filtered Delete: Same Operation, Different Syntax

Pinecone	<code>index.delete(filter={"source_doc_id": {"\$eq": "doc-42"}})</code>
pgvector	<code>DELETE FROM documents WHERE source_doc_id = 'doc-42';</code>
ChromaDB	<code>collection.delete(where={"source_doc_id": {"\$eq": "doc-42"}})</code>
Qdrant	<code>client.delete(collection_name="docs", points_selector=models.FilterSelector(filter=...))</code>
FAISS	<code># No metadata — maintain external ID mapping yourself ids = lookup_ids("doc-42"); index.remove_ids(np.array(ids))</code>

06: Hybrid Search and Reranking

Hybrid Search: Keyword + Semantic, Together

- Vector search misses exact keyword matches: product names, error codes, legal citations.
- Hybrid search combines keyword (BM25) with vector retrieval to cover both.

Product	Hybrid	How It Works
Weaviate	☑ Best-in-class	BM25 + vector fused with tunable alpha parameter
Pinecone	☑ Strong	Sparse + dense vectors in the same index
Qdrant	☑ Strong	Sparse vectors as first-class citizen type
pgvector	☑ Manual	Combine tsvector full-text search with vector similarity in SQL
ChromaDB	✗ No	Vector similarity + metadata filtering only
FAISS	✗ No	Pure vector. Build hybrid retrieval yourself.

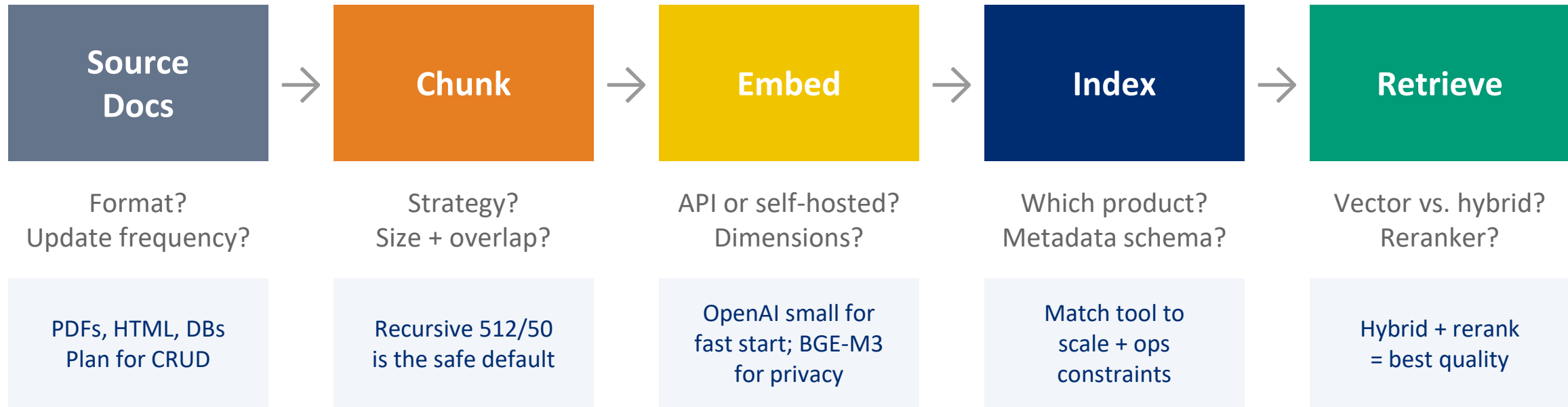
Reranking: The Second-Stage Quality Boost

Retrieve top-50 with hybrid search, then use a cross-encoder re-ranker (Cohere rerank-v3) to reorder by deep relevance. Typically, the single largest quality improvement is after chunking.
Cost: ~\$2 per 1,000 searches.

07: Putting It Together

The Pipeline and the Pitfalls

The Vector Search Pipeline



Each stage has its own failure modes. Debug retrieval quality by testing each stage independently.

Three Pitfalls That Burn Teams

ChromaDB-to-Production Cliff

Team builds in ChromaDB, hits scale limits, discovers migration to production DB requires rewriting all queries and re-embedding all data.

Fix: If you know you're going to production, prototype in your production tool.

Cost Surprise at Scale

Free-tier prototype works great. Production deployment at 50M vectors with 100K queries/day produces a monthly bill 6× the budget.

Fix: Model your cost at 10× current scale before committing to a platform.

Metadata Neglect

Vectors inserted with no source tracking. When documents change, no way to identify and delete stale vectors. Index accumulates ghost data.

Fix: Mandatory metadata from day one: source_doc_id, chunk_index, timestamps, tenant_id.

Common Scenarios → Product Picks

Weekend hackathon, building a chatbot



ChromaDB

Startup RAG product, small team, ship fast



Pinecone

200M product embeddings, dedicated search team



FAISS + GPU

Postgres shop adding semantic search



pgvector

Multi-tenant SaaS, strict data isolation



Qdrant

Keyword + semantic search across text + images



Weaviate

Decision Cheat Sheet

If you need...	Reach for...	Because...
Fastest prototype	ChromaDB	pip install and 3 lines of code
Zero ops, production-ready	Pinecone	Fully managed, auto-scales, no infra
Already on PostgreSQL	pgvector	One database, zero extra ops
Hybrid search + multi-modal	Weaviate	Native BM25 + vector, module ecosystem
Best filtering + multi-tenant	Qdrant	Most expressive filter language, Rust speed
Billion-scale + GPU	FAISS	Raw library performance, you build the rest
Cheapest (with infra team)	Qdrant or pgvector self-hosted	Open-source on your hardware
Cheapest (without infra team)	Pinecone (model the bill first!)	Managed, but watch cost at scale

What to Remember

It's a Spectrum, Not a Ranking

Six tools for different needs. ChromaDB isn't worse than FAISS — it's for a different use case. Match the tool to your constraints.

Embeddings & Chunking Outweigh the Database

Same corpus, chunked differently → larger retrieval quality delta than switching databases. Get preprocessing right first.

Design Your Metadata for Day 1000

source_doc_id, chunk_index, timestamps, embedding_model, tenant_id — mandatory from day one. Without it, updates are impossible.

The five decision axes are the durable skills. Products will change. The framework persists.

Copyright 2026. All rights reserved.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.